

Machine Learning Assignment 1

Course: 10 ECTS

Group: 06

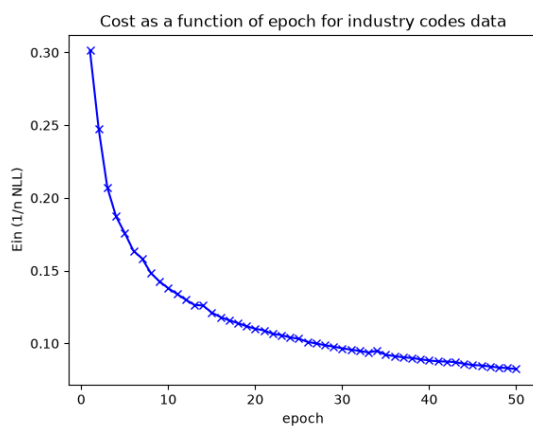
Jeppe Vestergaard Nielsen
Andreas Merrild Rasmussen
Jonas Hasring

202208635
202405123
202408369

Part 1: Logistic regression

Code

Summary & Results



In Sample Score (accuracy): 0.9730129390018484

Test Score (accuracy): 0.958980044345898

Actual code

CostGrad

```
### YOUR CODE HERE
margin = y * (X @ w)
cost = np.mean(np.logaddexp(0, -margin))
factor = np.empty_like(margin, dtype=float)
positive = margin >= 0
factor[positive] = np.exp(-margin[positive]) / (
    1 + np.exp(-margin[positive])
)
factor[~positive] = 1 / (1 + np.exp(margin[~positive]))
grad = -np.mean(y[:, None] * X * factor[:, None], axis=0)
### END CODE
assert grad.shape == w.shape
return cost, grad
```

Fit

```
if w is None: w = np.zeros(X.shape[1])
history = []
### YOUR CODE HERE
for _ in range(epochs):
    permutation = np.random.permutation(X.shape[0])
```

```

X_shuffled = X[permutation]
y_shuffled = y[permutation]

for start in range(0, X.shape[0], batch_size):
    end = start + batch_size
    X_batch = X_shuffled[start:end]
    y_batch = y_shuffled[start:end]
    _, grad = self.cost_grad(X_batch, y_batch, w)
    w -= lr * grad

cost, _ = self.cost_grad(X, y, w)
history.append(cost)
### END CODE
self.w = w
self.history = history

```

Theory

What is the running time of our mini-batch gradient descent algorithm?

Given n is the number of training data, d is the number of features, b is the mini-batch size, E is the number of Epochs

We have that one mini-batch will take $O(bd)$.

This we will multiply by the number of mini-batches per Epochs ($\frac{n}{b}$)

$$\frac{n}{b} * O(bd) = O(nd)$$

This will then be multiplied by the total number of Epochs and result in

$$O(E * nd)$$

Which is the runtime of our mini batch gradient descent algorithm.

What happens if we randomly permute pixels in training images, before we train?

Permuting the data only changes which feature corresponds to which pixel position. This means we don't lose any information from permuting the data. In fact, there is a one to one correspondence between a classifier on the original data and one on the new data, with same predictions and loss, meaning performance stays the same.

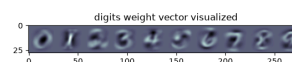
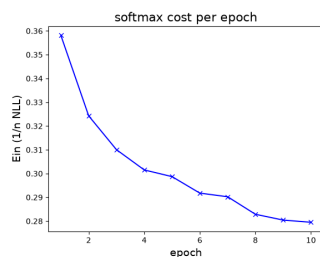
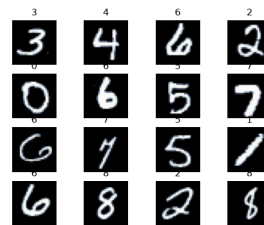
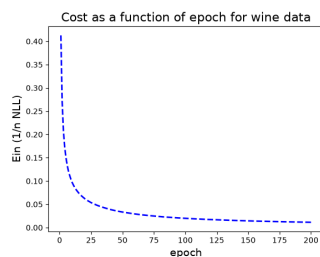
If the data is linearly separable what happens to weights when we implement logistic regression with gradient descent?

For linearly separable data, logistic regression has no finite minimizer. Gradient descent drives the logistic loss toward zero by continually increasing the magnitude of the weight vector. Therefore $\|w_t\| \rightarrow \infty$ While the norm grows without bound, the direction of the weight vector converges toward the maximum-margin separating hyperplane. Thus the weights become arbitrarily large and the training loss approaches zero.

Part 2: Softmax

Code

Summary & Results



For the -wine test we have these:

In Sample Score (accuracy): 1.0

Test Score (accuracy): 0.9777777777777777

Actual code

CostGrad

```
cost = np.nan
grad = np.zeros(W.shape)*np.nan
Yk = one_in_k_encoding(y, self.num_classes) # may help - otherwise you may
remove it
### YOUR CODE HERE
n = X.shape[0]
scores = X @ W
probs = softmax(scores)
cost = -np.mean(np.sum(Yk * np.log(probs), axis=1))
grad = - (X.T @ (Yk - probs)) / n
### END CODE
return cost, grad
```

Fit

```
if W is None: W = np.zeros((X.shape[1], self.num_classes))
history = []
### YOUR CODE HERE
n = X.shape[0]
batch_size = min(batch_size, n)

for _ in range(epochs):
    indices = np.random.permutation(n)

    for start in range(0, n, batch_size):
        batch_indices = indices[start:start + batch_size]
        _, grad = self.cost_grad(
```

```

        X[batch_indices], Y[batch_indices], W
    )
    W -= lr * grad

    epoch_cost, _ = self.cost_grad(X, Y, W)
    history.append(epoch_cost)
### END CODE
self.W = W
self.history = history

```

Theory

What is the running time of our softmax implementation?

The softmax implementation works for K classes, instead of simply being for two classes like the logistic one. This means that for each batch W is shape $d \times K$. This means the cost of matrix multiplication becomes $O(bdK)$. Performing the same steps as for logistic regression means for all batches it becomes $O(ndK)$, and then taking epochs into account the total cost is $O(End * K)$